# Quality Management

IT Product Development

Innopolis University, 2026

# Agenda

- Quality management
- Quality assurance
- Quality requirement tests
- Deployment, environments, automation

# Quality Management

# Quiz

- What can you compete on when other products can copy all of your features using swarms of LLM-powered agents?

  Example:

  PhotoCraft - Image editing; an open-source, clean-room reimplementation of Adobe Photoshop, rebuilt in pure Rust.

  Layers, masks, adjustment layers, layer styles, type, vectors, brushes and real PSD files, in a native app written entirely in Rust. Open source, offline, and yours.

# Quality management - definition

From Quality management, ISO 9000:2026:

**Quality management (QM)** ensures that an organization, product, or service consistently performs as intended.

It has four main components:

- **quality planning** - setting quality objectives and specifying processes necessary for providing products and services, and related sources to achieve quality objectives.
- **quality assurance** - providing confidence that quality requirements will be fulfilled
- **quality control** - fulfilling quality requirements
- **quality improvement** - increasing the ability to fulfill quality requirements

Customers recognize that quality is an important attribute when choosing and purchasing products and services. Suppliers can recognize that quality is an important differentiator of their offerings, and endeavor to **compete on the quality** of their products and the service they offer.

# Quality management in Industrial Project and ITPD

From Practice areas:

> “Teams are able to show that they have established quality mechanisms consistent with the needs of their client. Quality mechanisms may include prevention mechanisms as well as detection mechanisms. Teams will collect and have available basic quality plans and metrics consistent with the phase of the project.”

# Key takeaways

- Products can compete on quality
- Quality activities should be planned

# Quality assurance

# Quiz

- When is the software ready for a release?
- When is the quality good enough?

# Verification and validation

From ISO 9000:2026:

**Verification** - confirmation, through the provision of objective evidence, that specified requirements have been fulfilled.

**Validation** - confirmation, through the provision of objective evidence, that the requirements for a specific intended use or application have been fulfilled.

From Software Quality and Testing, Lecture 1:

**Verification** - Proving that software conforms to its functional and non-functional requirements. Shows that software is “high quality”.

**Validation** - Proving that software meets customer’s true requirements, needs, and expectations. Shows that software is actually useful (harder!).

# Verification examples

- Design can be verified against requirements
- Code can be verified against the design and requirements
- Processes can be verified against compliance requirements
- Agent behavior can be verified against the guardrails
- The system behavior can be verified against a formal model (example)

# Quality gates

From Quality gate

> A quality gate is a configurable checkpoint in the software development lifecycle that enforces predefined standards before code can advance to the next stage.

V in ETVX (Entry, Task, Validation/Verification, Exit)

Example of quality gates for a PR (current stage: submitted; next stage: merged):

- All links resolved
- No Markdown linter warnings or errors
- All unit tests passed
- All acceptance criteria satisfied

# Key takeaways

ITPD focus:

- Manual verification of changes against acceptance criteria
- Automated verification of code via tests
- Validation through regular meetings with the customer

# Quality requirement tests

# Quiz

- How to prove that the system satisfies a quality requirement?
- Which quality checks to implement first - the easiest ones or the most important ones?

# The ISO 25010 Quality Model

[Iso25010]:

Quality → Characteristics → Sub-characteristics

| SOFTWARE PRODUCT QUALITY | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| FUNCTIONAL SUITABILITY | PERFORMANCE EFFICIENCY | COMPATIBILITY | INTERACTION CAPABILITY | RELIABILITY | SECURITY | MAINTAINABILITY | FLEXIBILITY | SAFETY |
| FUNCTIONAL COMPLETENESS<br>FUNCTIONAL CORRECTNESS<br>FUNCTIONAL APPROPRIATENESS<br>iso25000.com | TIME BEHAVIOUR<br>RESOURCE UTILIZATION<br>CAPACITY | CO-EXISTENCE<br>INTEROPERABILITY | APPROPRIATENESS RECOGNIZABILITY<br>LEARNABILITY<br>OPERABILITY<br>USER ERROR PROTECTION<br>USER ENGAGEMENT<br>INCLUSIVITY<br>USER ASSISTANCE<br>SELF-DESCRIPTIVENESS | FAULTLESSNESS<br>AVAILABILITY<br>FAULT TOLERANCE<br>RECOVERABILITY | CONFIDENTIALITY<br>INTEGRITY<br>NON-REPUDIATION<br>ACCOUNTABILITY<br>AUTHENTICITY<br>RESISTANCE | MODULARITY<br>REUSABILITY<br>ANALYSABILITY<br>MODIFIABILITY<br>TESTABILITY | ADAPTABILITY<br>SCALABILITY<br>INSTALLABILITY<br>REPLACEABILITY | OPERATIONAL CONSTRAINT<br>RISK IDENTIFICATION<br>FAIL SAFE<br>HAZARD WARNING<br>SAFE INTEGRATION |

# Example: A QR for Moodle's Availability

- Business importance: High
- Technical risk: High

| | |
|---|---|
| **Source of Stimulus** | External: A large number of students (e.g., 500+) simultaneously using the system. |
| **Stimulus** | Fault (Crash): An unexpected number of concurrent database write operations, triggered by all students submitting a high-stakes quiz in the final minute before the deadline, causes the primary database server to crash. |
| **Artifact** | The Moodle application, specifically the database server and the services that depend on it (e.g., quiz submission, gradebook, session management). |
| **Environment** | Normal (Peak) Operation: The system is operating under a heavy but expected load on a weekday evening, which is a known peak usage time for students. |
| **Response** | The system’s high-availability mechanism detects the primary database failure. It automatically initiates a failover to a redundant, standby database server. A brief maintenance page is displayed to users currently in the process of submitting, and the system then seamlessly restores their sessions once the failover is complete. |
| **Response Measure** | Full service is restored within 3 minutes. No more than 0.5% of student quiz submissions are lost and require resubmission. The system’s availability must not drop below 99.9% during the academic term. |

# Example: A QR test

- **Step 1: Simulate the Stimulus**
  - In a staging environment, we use a load testing tool (JMeter) to simulate **500+ students** submitting a quiz at the exact same second.
- **Step 2: Inject the Fault**
  - At the precise moment of peak submission load, we intentionally **crash the primary database server** to trigger the automated failover system.
- **Step 3: Measure the Response**
  - The test is considered a **PASS** only if the system automatically recovers and meets these strict criteria:
    - **Recovery Time:** Full functionality is restored in **under 3 minutes**.
    - **Data Integrity:** Fewer than **0.5%** of student submissions are lost.
    - **Availability:** The overall uptime of **99.9%** is maintained.

# Key takeaways

- Quality requirements should be testable
- Prove their testability by implementing QRTs
- Implementing QRTs requires effort which should be justified
- Implement QRTs for the most important quality requirements first
- Negotiate the priorities with the customer

# Deployment, environments, automation

# Quiz

- How to reduce effort required for quality assurance?

# Environments

[SoftArchPrac]:

**Deployment** is a process that starts with coding and ends with real users interacting with the system in a **production environment**.

**Development environment** - Where code for your service is developed and submitted to a version control system. Code is usually tested locally (e.g. via unit tests).

**Integration environment** - Where an executable version of the service is built from the existing, changed, and new code. Unit and integration tests can be run against the built version.

**Staging environment** - Where other quality requirements are tested. This environment should represent the production environment well enough so that if quality requirement tests pass in this environment, the confidence is high that they would pass in the production environment.

**Production environment** - Where the service is monitored closely until all parties have some level of confidence in its quality.

# Automation

[SoftArchPrac]:

**Continuous integration** - automated running of tests in the integration environment.

**Continuous deployment** - fully automated deployment (without human intervention).

**Continuous delivery** - automated deployment with deliberate gates requiring human intervention (perhaps due to regulations or policies or constraints).

# Key takeaways

- Quality management lets you build systems that behave as intended in production
- Automating quality activities helps you reduce effort to achieve desired quality requirements

# References


1. [Iso25010]: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010
2. [SoftArchPrac]: L. Bass, P. Clements, and R. Kazman, Software Architecture in Practice, 4th ed.